

13

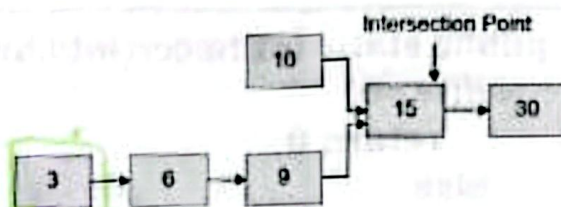
1. (20 puntos) Conceptos

a. (20 puntos) Mapee la primera columna con la segunda columna:

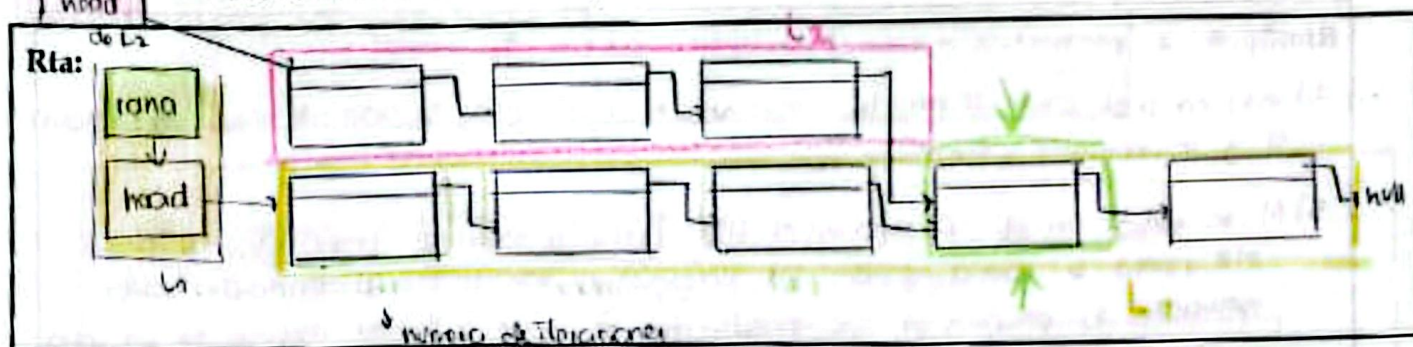
1. Método push()	(7) Saca un elemento de la cola
2. Método pop()	(1) Inserta un elemento a la pila
3. Método peek()	(3) Retorna el dato sin eliminarlo
4. Aplicación: historial de navegación	(10) Relación de recurrencia de Fibonacci
5. $F_n = n F_{n-1}$	(7) Estructura de datos lista enlazada
6. $a_n = na_{n-2}$	(6) Relación de recurrencia del Factorial
7. Aplicación: Lista de reproducción	(4) Estructura de datos tipo cola
8. Método enqueue()	(2) Saca un elemento de la pila (el de la cima)
9. Método dequeue()	(8) Inserta un elemento en la cola
10. $F_n = F_{n-1} + F_{n-2}$	(5) Relación de recurrencia homogénea

2. (20 puntos) Listas enlazadas simples

Por algún error de programación, dos listas se enlazan en algún nodo llamado *nodo intersección* como se muestra en la siguiente figura.



- a. (8 puntos) Explicar con diagramas, cómo se puede buscar el punto o nodo intersección de las dos listas enlazadas simples.



- b. (12 puntos) Escribir el código en Java del método propuesto para buscar el nodo intersección.

se piden las listas para poder recorrerlas

Rta: `public void buscarIntersectionPoint (Lista L1, Lista L2) {`

```

    Nodo rana = head;
    int counter = 0;
    while (rana != null) {
        while ((L1.rana.datum != L2.rana.datum) && (L1.rana.next != null)) {
            counter++;
            rana = rana.next;
        }
    }

```

```

    System.out.println("la posición es: " + counter + " con el dato: " + rana.datum);
    return;
}

```

(1) Se compara si los datos en esa posición son los mismos, ya que si tienen un "intersection point" deberán desde ese punto empezar a tener los mismos datos pero por complicación se analiza el siguiente al nodo auxiliar en nuestro caso es rana.

3. (20 puntos) Recursión.

- a. (12 puntos) Dado el siguiente método recursivo **hacer**. ¿Describa con sus palabras, qué procedimiento realiza?

1	public static int hacer(int i, int j){	1, 2.
2	if(i==0)	$(\frac{1}{2}, 2) + 1$
3	return 0;	
4	else	
5	return hacer(i/j, j) + 1;	
6	}	

Rta: pide 2 parámetros enteros para operar dentro del método.

2) Verifica si el primer parámetro ingresado es igual a 0, si entra a la condición retorna 0 y se sale del método.

5) Al no entrar en el if, sino en el else vuelve a llamarse para retornar en el else, pero se llama usando los parámetros ingresados al principio de la ecuación, lo peculiar es que igual, como se vuelve a llamar verifica que no vaya a dar 0 el primer parámetro al ejecutarse, ya que de lo contrario i/j da 0 de una, si llega a dar 0 o cercano es porque se dieron las iteraciones suficientes para salirse del método.

(4)

- b. (8 puntos) ¿Qué retorna el método si se pasan los siguientes parámetros: **hacer(82,3)**, muestre el procedimiento? Nota: Una prueba de escritorio podría ser útil para encontrar la solución.

Rta:

82, 3 $3,7 \approx 4$ → al ser decimal y no entero va a aproximar

$\text{return}(82/3, 3) + 1 = (3,7, 4) \rightarrow 1 \text{ iteración}$

$(4/4, 4) + 1 = (1, 4) = (1,9, 4) \rightarrow \text{pero al ser}$

(2)

4. (20 puntos) Colas.

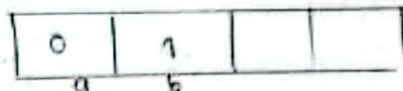
- a. (10 puntos) Se tiene el siguiente método **hacer**. ¿Describe con sus palabras, qué operación realiza el método?

```

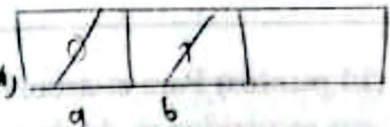
1 public static void hacer(int n){
2     Queue<Integer> cola = new LinkedList<Integer>();
3     cola.add(0);
4     cola.add(1);
5     for (int i = 0; i < n; i++){
6         int a = cola.poll();
7         int b = cola.poll();
8         cola.add(b);
9         cola.add(a + b);
10        System.out.println(a);
11    }
12 }

```

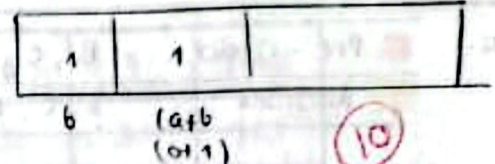
① Se agregan 2 números a la cola (0 y 1).



② Se va a iterar n veces el siguiente procedimiento:
para una variable entera a se elimina el dato de la cola pero a guarda el valor b mismo para b .



③ Se añade a la cola el valor de b y se añade en la segunda casilla $(a+b)$.

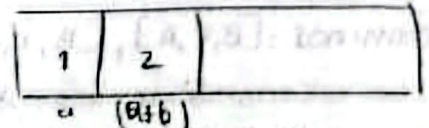


④ Se imprime $a \rightarrow a=0$

[0,]

para la segunda iteración

a va a guardar el valor de la casilla 1. $\rightarrow a=1$.



El método imprime fibonacci hasta la longitud indicada en el n para retro

- b. (10 puntos) ¿Cuál es el resultado de ejecutar el método **hacer()** para $n=3$ y $n=6$?

Para 3: [0, 1, 2]

Para 6: [0, 1, 2, 3, 5, 8]

5. (20 puntos) Árboles binarios

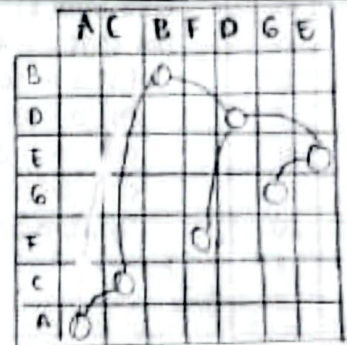
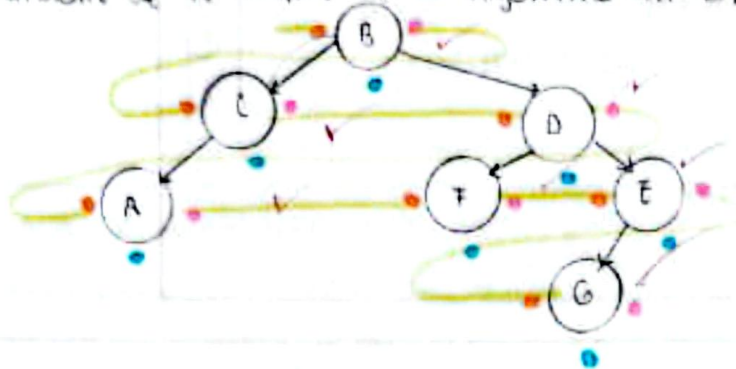
- a. (10 puntos) Dibuje el árbol binario con nodos A, B, C, D, E, F, G, al cual le corresponden los siguientes recorridos:

In-order: A, C, B, F, D, G, E →

Post-order: A, C, F, G, E, D, B ↑

post: a, a, a
Pre (1, r, d)
bas (1, d, 1)

Rta: guíame de la matriz y el algoritmo In-order



Hay una manera de hacerlo por medio de la matriz de adyacencia, da un aproximado de la creación de los nodos en el árbol.

Interesante

- b. (10 puntos) Para el árbol binario resultante del ítem (a), calcular: altura, peso, caminos, y sus recorridos en Anchura y en Pre-orden.

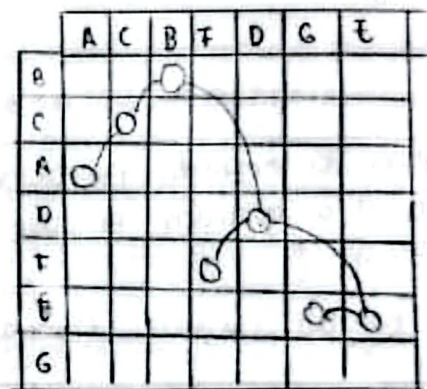
Rta: Pre-order: B, C, A, D, F, E, G ✓
Anchura: B, C, D, A, F, E, G ✓

comprobar que si es este el orden en la matriz de adyacencia.

Altura: (Nivel + 1) = Nivel + 3 → Altura: 4 ✓

Peso: 7 ✓

Caminos: [B, C, A], [B, D, F], [B, D, E, G] ✓ ✓ ✓



pre ↓ post ↑

10